Part II: Graph Algorithms

Lecture 7: Minimum Spanning Trees and Prim's Algorithm

Computer Science and Technology
United International College

Objective and Outline

Objective: Introduce a second problem on graphs, i.e. MST, and discuss one solution.
Reference: Chapter 23 of CLRS

1. Spanning trees and minimum spanning trees (MST)
2. Strategy for solving the MST problem
3. Prim's algorithm for the MST problem.
   - The idea
   - The algorithm
   - Analysis

Spanning Trees I

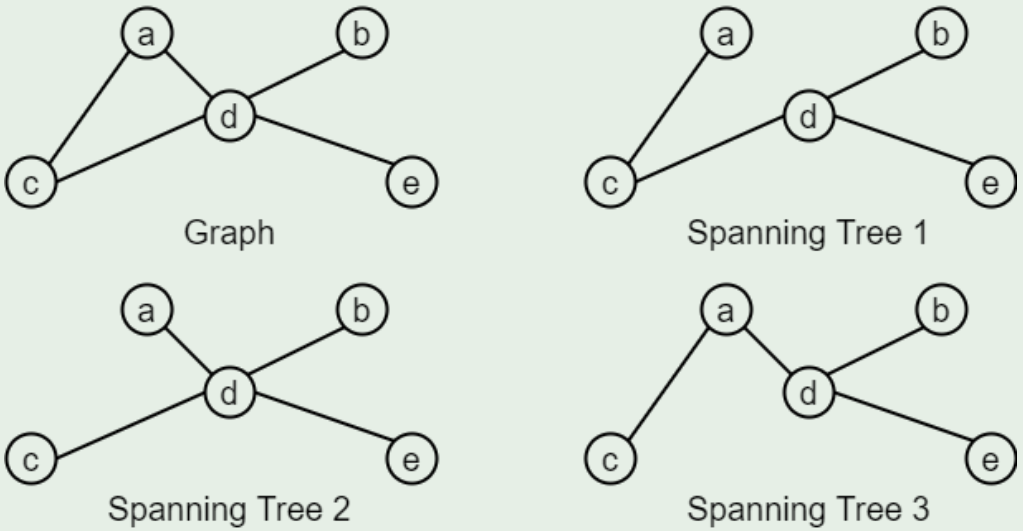
Definition (Subgraph)

$G' = (V', E')$ is a subgraph of an undirected graph $G = (V, E)$ if $V' = V$ and $E' \subseteq E$, denoted as $G' \subseteq G$.

Definition (Spanning Tree)

The subgraph T of a connected graph $G = (V, E)$ is a spanning tree if T is a tree.

Example



Spanning Trees II

Theorem

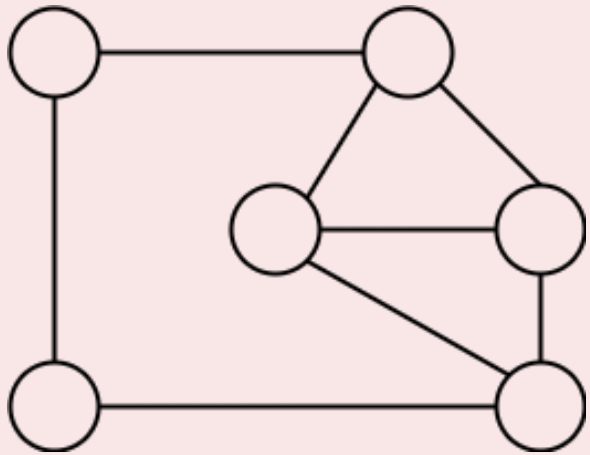
Every connected graph has a spanning tree.

Question

Why is this true?

Question

Given a connected graph G, how can you find a spanning tree of G?

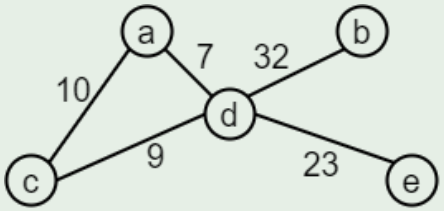


Weighted Graphs

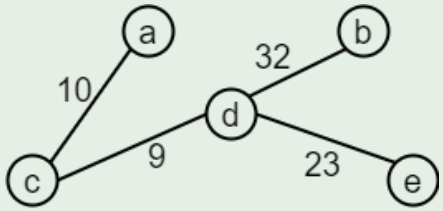
Definition

$G = (V, E)$ is weighted, if there is a weight function $w : E \to \mathbb{R}$ such that $w((u, v))$ is the weight of the edge $(u, v) \in E$.

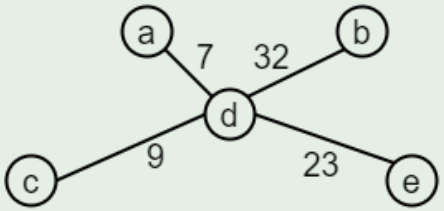
Example



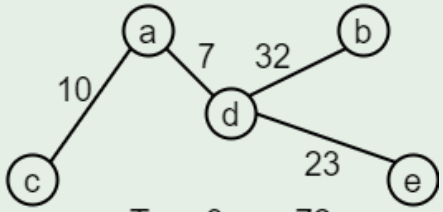
Weighted Graph



Tree 1, w = 74



Tree 2, w = 71



Tree 3,w = 72

Definition

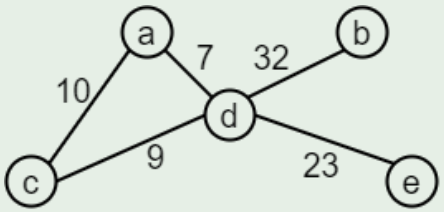
The weight of a graph (subject to w) is $w(G) = \sum_{(u,v)\in E} w(u, v)$.

Minimum Spanning Trees

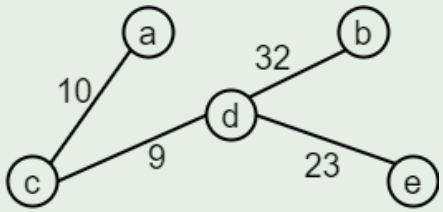
Definition (Minimum Spanning Tree)

T is the Minimum spanning tree of G if T is a spanning tree of G and $w(T)$ is minimized (among all spanning trees of G).

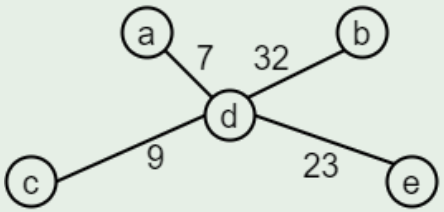
Example



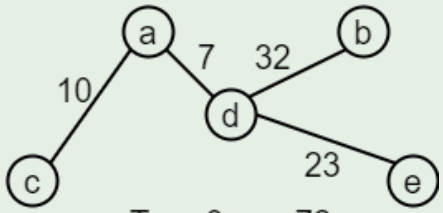
Weighted Graph



Tree 1, w = 74



Tree 2, w = 71

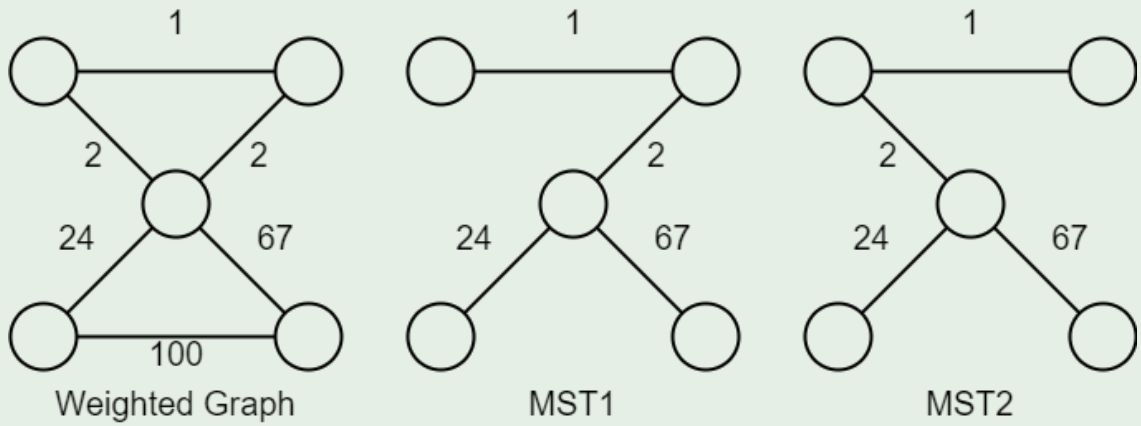


Tree 3,w = 72

Remark

The minimum spanning tree may not be unique.

Example



However,

- the weight of MSTs is unique; and
- if the weights of all the edges are distinct, MST is indeed unique (we won't prove this now).

Minimum Spanning Tree Problem

Definition (MST Problem)

Given a connected weighted undirected graph G, find a minimum spanning tree (MST) of G.

Outline

General strategy for solving the MST Problem

A tree is an acyclic graph.

- Start with $(V, \emptyset)$, an empty graph.
- Try to add one edge at a time, always making sure that what is built remains acyclic.
- If we are sure every time the resulting graph is always a subset of some minimum spanning tree, we are done.

Generic Algorithm for MST problem

Definition (Safe Edge)

Let A be a set of edges such that $A \subseteq T$,where T is a MST.An edge (u, v) is a safe edge for A, if $A \cup (u, v)$ is also a subset of some MST.

- If at each step, we can find a safe edge (u, v), we can grow a MST.

Algorithm 1 *GenericMST*(G, w)

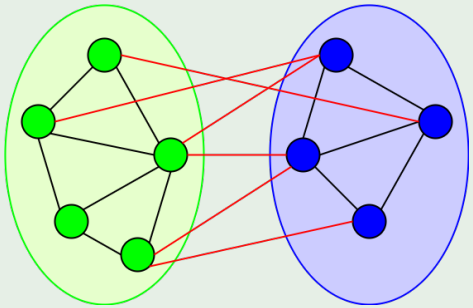
1: $A = EMPTY$
2: **while** A does not form a spanning tree **do**
3: find an edge (u, v) that is safe for A
4: add (u, v) to A
5: **end while**
6: **return** A

Some Definitions

Definition (Cut)

Let $G = (V, E)$ be a connected and undirected graph. A cut $(S, V \setminus S)$ of G is a partition of V.

Example



Note:

- "$V \setminus S$" reads "V set minus S".
- (V_1, V_2) is a partition of V if $V_1 \cap V_2 = \emptyset$ and $V_1 \cup V_2 = V$.

Some Definitions

Definition (Cross)

An edge $(u, v) \in E$ crosses the cut $(S, V \setminus S)$ if $u \in S$ and $v \in V \setminus S$.

Definition (Respect)

A cut respects a set A of edges if no edge in A crosses the cut.

Definition (Light Edge)

An edge is a light edge crossing a cut if its weight is the minimum of any edge crossing the cut.

How to Find a Safe Edge? I

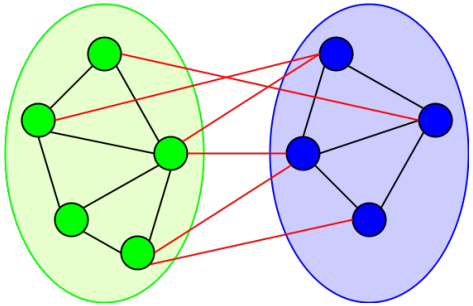
Lemma

Let

- $G = (V, E)$ be a connected, undirected graph with a real-valued weight function w defined on E.
- A be a subset of E that A is included in some minimum spanning tree of G.
- $(S, V \setminus S)$ be any cut of G that respects A.
- (u, v) be a light edge crossing the cut $(S, V \setminus S)$.

Then, edge (u, v) is safe for A.

How to Find a Safe Edge? II



It means that we can find a safe edge by

1. first finding a cut that respects A,
2. then finding the light edge crossing that cut.

That light edge is a safe edge.

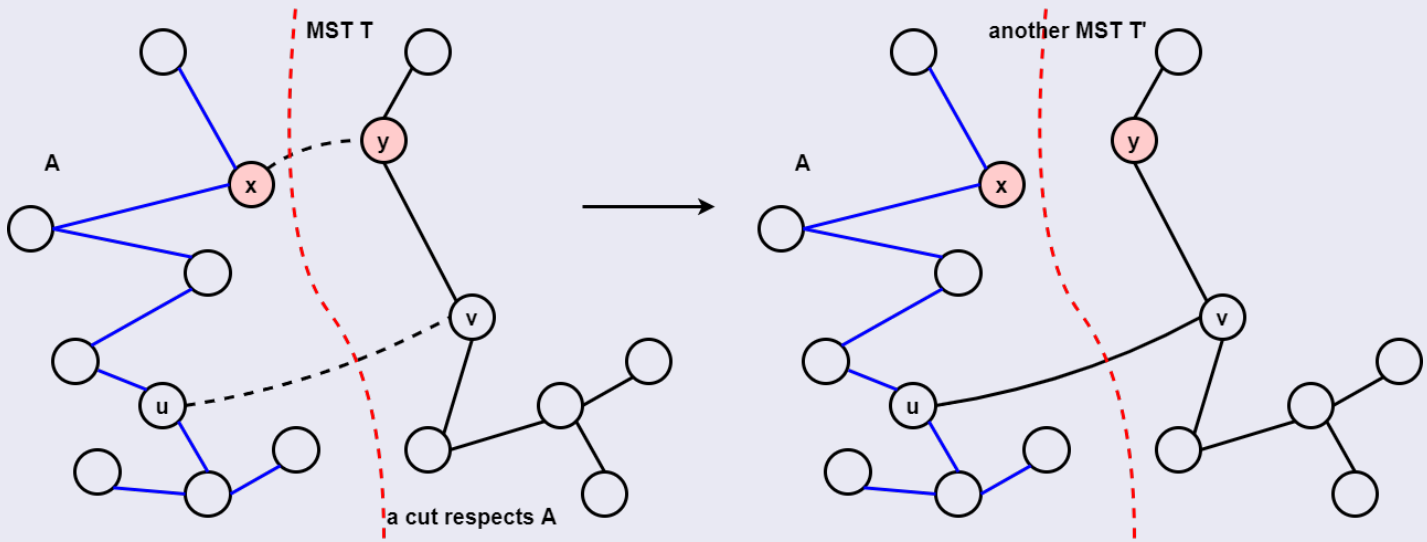
Proof.

- Let $A \subseteq T$,where T is a MST.
- Case1: $(u, v) \in T$
  - $A \cup \{(u, v)\} \subseteq T$.
  - Hence (u, v) is safe for A.

How to Find a Safe Edge? III

Proof.(Con't)

- Case2: $(u, v) \notin T$
  - Idea: construct another MST T' s.t. $A \cup \{(u, v)\} \subseteq T'$.
  - Consider a path P in T from u to v.
  - Since u and v are on opposite sides of the cut $(S, V \setminus S)$,
    - There is at least one edge in P that crosses the cut.
    - Let (x, y) be such edge.



How to Find a Safe Edge? IV

Proof.(Con't)

- Since the cut respects A, $(x, y) \notin A$.
- Since (u, v) is a light edge crossing the cut, we have $w(x, y) \geq w(u, v)$.
- Add (u, v) to T, it creates a cycle. By removing an edge from the cycle, it becomes a tree again. In particular, we remove $(x, y)(\notin A)$ to make a new tree T'.
- The weight of T' is

$$w(T') = w(T) - w(x, y) + w(u, v) \leq w(T)$$

- Since T is a MST, we must have $w(T) = w(T')$, hence T' is also a MST.
- Since $A \cup \{(u, v)\} \subseteq T'$, (u, v) is safe for A.
- The Lemma is proved.

Outline

Prim's Algorithm

The generic algorithm gives us an idea how to 'grow' a MST.

- If you read the theorem and the proof carefully, you will notice that the choice of a cut (and hence the corresponding light edge) in each iteration is immaterial.
- We can select any cut (that respects the selected edges) and find the light edge crossing that cut to proceed.

Prim's algorithm makes a nature choice of the cut in each iteration.

- It grows a single tree and adds a light edge in each iteration.

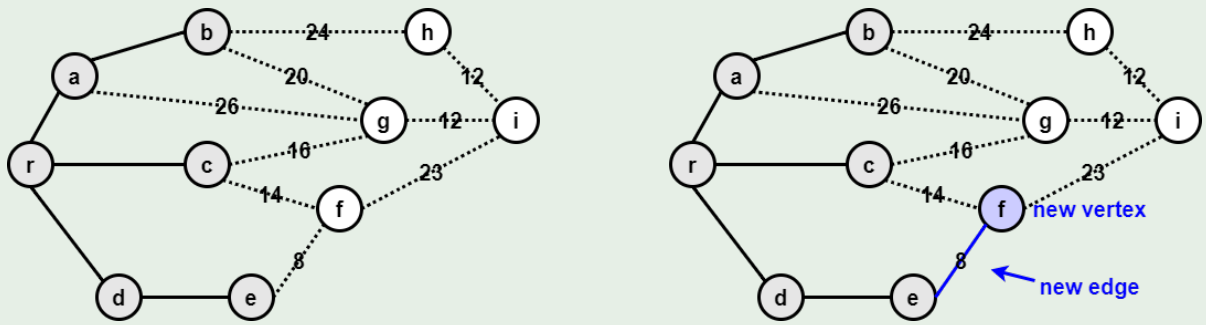
To grow a tree,

- start by picking any vertex r to be the root of the tree;
- while the tree does not contain all vertices in the graph: find shortest edge leaving the tree and add it to the tree.

We will show that these steps can be implemented such that the algorithm's running time is $\mathcal{O}(|E| \log |V|)$.

More Details

Example



Step 0:

- Choose an arbitrary vertex r. Initialize $S = \{r\}$ and $A = \emptyset$.
- r will be the root of the spanning tree.

Step 1:

- Find a lightest edge such that one endpoint is in S and the other is in $V \setminus S$.
- Add this edge to A and its (other) endpoint to S.

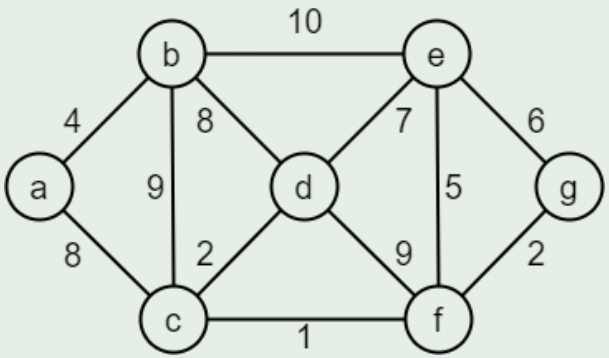
Step 2:

- If $V \setminus S = \emptyset$, then stop and output (minimum) spanning tree (S, A); Otherwise, repeat Step 1.

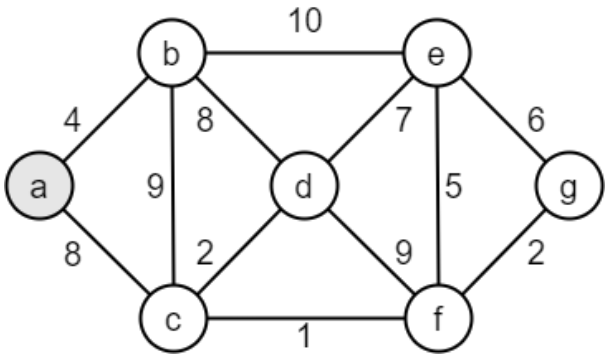
Prim's Example I

Example

Connected weighted undirected graph

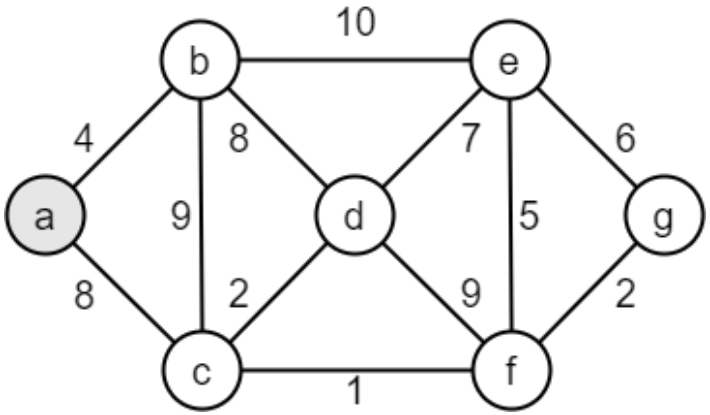


0. Initialization
   $S = \{a\}$, $V \setminus S = \{b, c, d, e, f, g\}$, the lightest edge is (a, b).

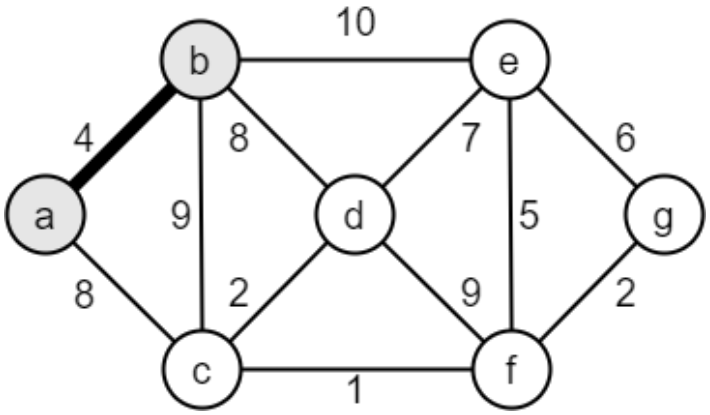


Prim's Example II

1.
   - Before iteration 1, $S = \{a\}$, $V \setminus S = \{b, c, d, e, f, g\}$, $A = \{\}$, the lightest edge is (a, b).

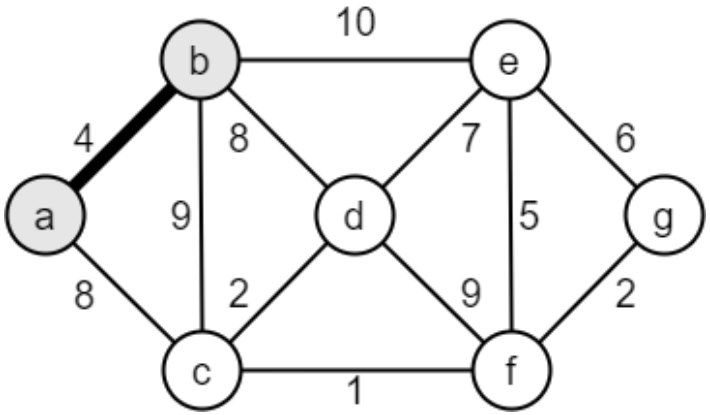


   - After iteration 1, $S = \{a, b\}$, $V \setminus S = \{c, d, e, f, g\}$, $A = \{(a, b)\}$, the lightest edges are (b, d), (a, c).

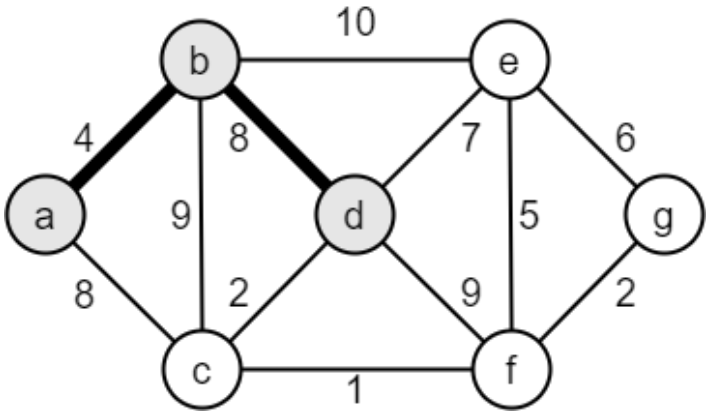


Prim's Example III

2.
   - Before iteration 2, $S = \{a, b\}$, $V \setminus S = \{c, d, e, f, g\}$, $A = \{(a, b)\}$, the lightest edges are (b, d), (a, c).

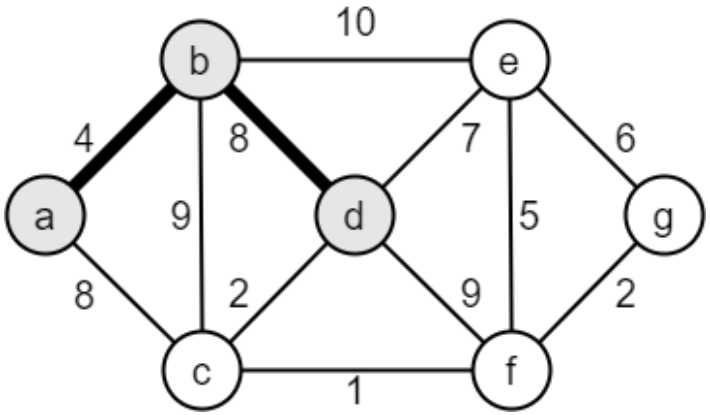


   - After iteration 2, $S = \{a, b, d\}$, $V \setminus S = \{c, e, f, g\}$, $A = \{(a, b), (b, d)\}$, the lightest edge is (d, c).

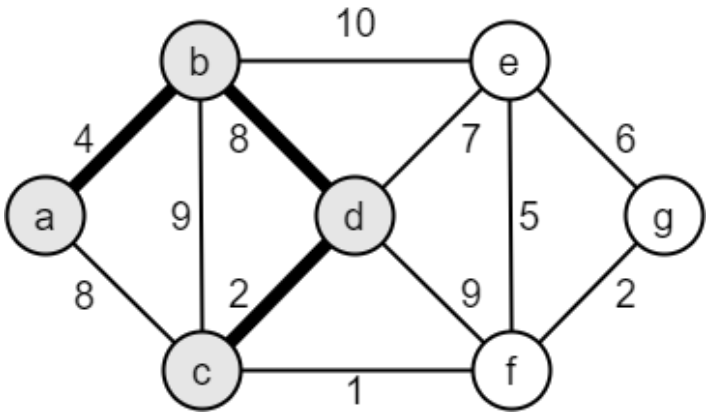


Prim's Example IV

3.
   - Before iteration 3, $S = \{a, b, d\}$, $V \setminus S = \{c, e, f, g\}$, $A = \{(a, b), (b, d)\}$, the lightest edge is (d, c).

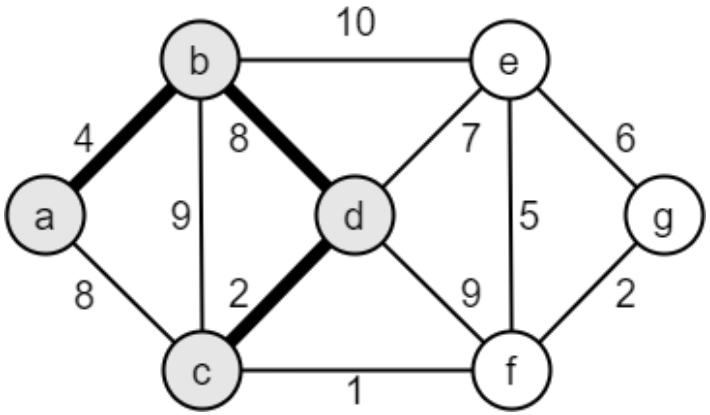


   - After iteration 3, $S = \{a, b, c, d\}$, $V \setminus S = \{e, f, g\}$, $A = \{(a, b), (b, d), (c, d)\}$, the lightest edge is (c, f).

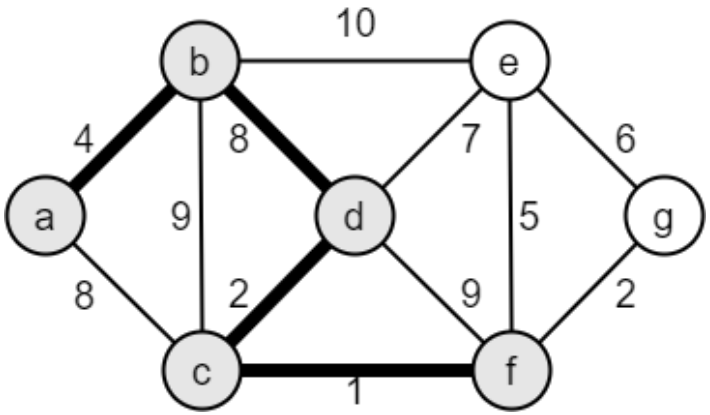


Prim's Example V

4.
   - Before iteration 4, $S = \{a, b, c, d\}$, $V \setminus S = \{e, f, g\}$, $A = \{(a, b), (b, d), (c, d)\}$, the lightest edge is (c, f).

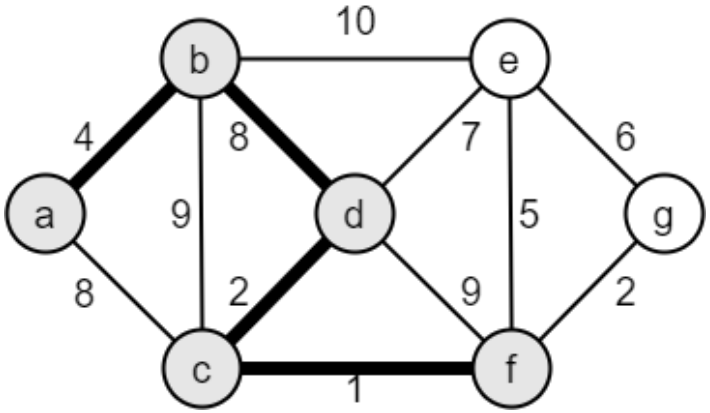


   - After iteration 4, $S = \{a, b, c, d, f\}$, $V \setminus S = \{e, g\}$, $A = \{(a, b), (b, d), (c, d), (c, f)\}$, the lightest edge is (f, g).

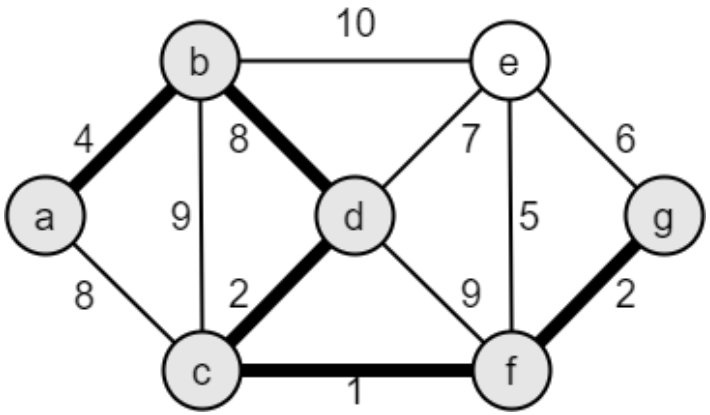


Prim's Example VI

5.
   - Before iteration 5, $S = \{a, b, c, d, f\}$, $V \setminus S = \{e, g\}$, $A = \{(a, b), (b, d), (c, d), (c, f)\}$, the lightest edge is (f, g).



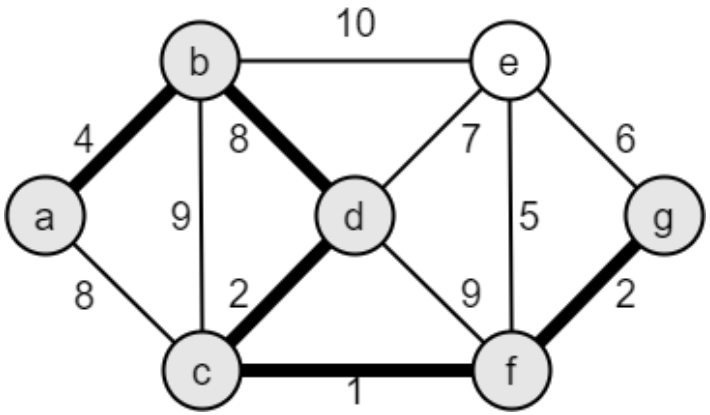
   - After iteration 5, $S = \{a, b, c, d, f, g\}$, $V \setminus S = \{e\}$, $A = \{(a, b), (b, d), (c, d), (c, f), (f, g)\}$, the lightest edge is (f, e).



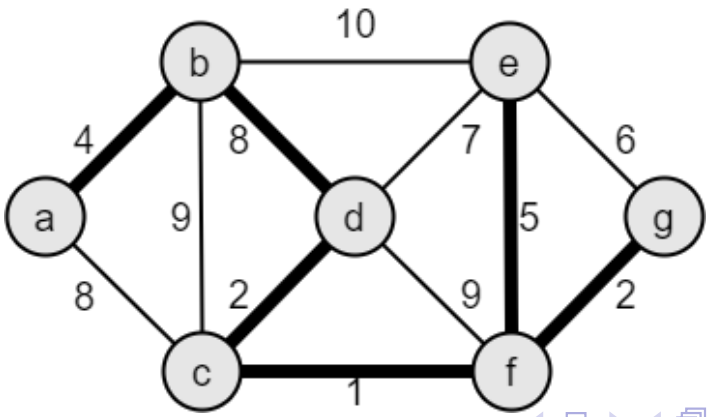
Prim's Example VII

6

- Before iteration 6, $S = \{a, b, c, d, f, g\}$, $V \setminus S = \{e\}$, $A = \{(a, b), (b, d), (c, d), (c, f), (f, g)\}$, the lightest edge is (f, e).



- After iteration 6, $S = \{a, b, c, d, e, f, g\}$, $V \setminus S = \{\}$, $A = \{(a, b), (b, d), (c, d), (c, f), (f, g), (f, e)\}$. MST is completed.



Outline

Recall Idea of Prim's Algorithm

Step 0:

- Choose an arbitrary vertex r. Initialize $S = \{r\}$ and $A = \emptyset$.
- r will be the root of the spanning tree.

Step 1:

- Find a lightest edge such that one endpoint is in S and the other is in $V \setminus S$.
- Add this edge to A and its (other) endpoint to S.

Step 2:

- If $V \setminus S = \emptyset$, then stop and output (minimum) spanning tree (S, A); Otherwise, repeat Step 1.

Questions

1. Why does this produce a minimum spanning tree?
2. How does the algorithm update S efficiently?
3. How does the algorithm find the lightest edge and update A efficiently?

Prim's Algorithm

Questions

How does the algorithm update S efficiently?

Answer: Color the vertices.

- Initially, all vertices are white.
- Change the color to black when the vertex is moved to S.
- Use $color[v]$ to store color.

Questions

How does the algorithm find the lightest edge and update A efficiently?

Answer:

- Use a priority queue to find the lightest edge.
- Use $pred[v]$ to update A.

Reviewing Priority Queues

Priority Queue is a data structure. It can be implemented as a heap.

A priority queue supports the following functions.

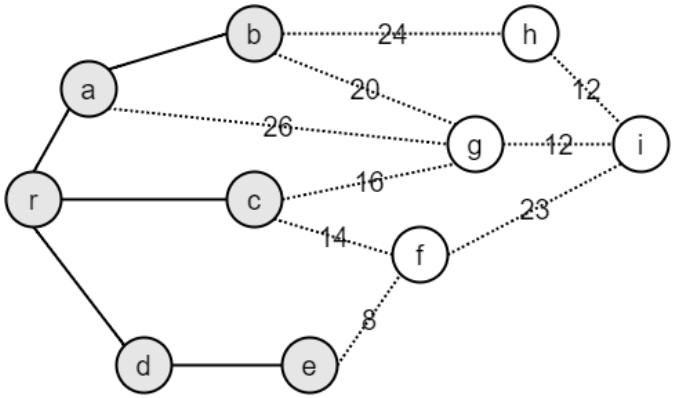
- $insert(u, key)$: Insert u with the key value key in Q.
- $u = extractMin()$: Extract the item with minimum key value in Q.
- $decreaseKey(u, NewKey)$: Decrease u's key value to $NewKey$.

Remark: Priority Queues can be implemented so that each operation takes time $\mathcal{O}(\log |Q|)$. See Chapter 6.5 CLRS.

Using a Priority Queue to Find the Lightest Edge

Each item of the queue is a triple $(u, pred[u], key[u])$, where

- u is a vertex in $V \setminus S$,
- $key[u]$ is the weight of the lightest edge from u to any vertex in S, and
- $pred[u]$ is the endpoint of this edge in S. (The array is used to build the MST tree.)



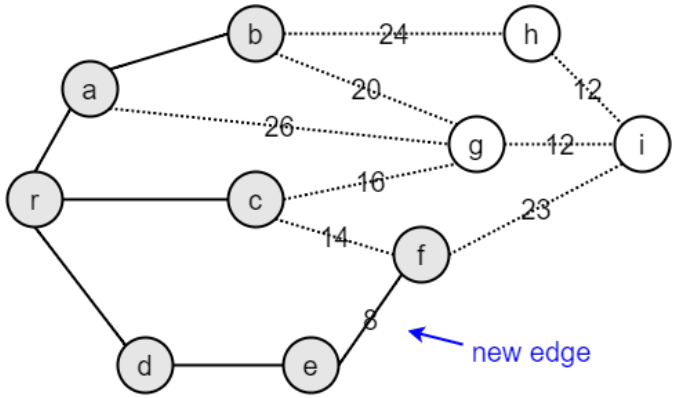
key[f] = 8, pred[f] = e

key[g] = 16, pred[g] = c

key[h] = 24, pred[h] = b

key[i] = infinity, pred[i] = nil

⟶ f has the minimum key



key[i] = 23, pred[i] = f

After adding the new edge and vertex f, update the key[v] and pred[v] for each vertex v adjacent to f

Description of Prim's Algorithm

Remark: G is given by adjacency lists. The vertices in $V \setminus S$ are stored in a priority queue with $key = value$ of lightest edge to vertex in S.

Pseudo Codes of Prim's Algorithm

Algorithm 2 $Prim(G, w, r)$

```
 1: for all u ∈ V do                                  // initialize
 2:     key[u] = ∞
 3:     color[u] = W
 4: end for
 5: key[r] = 0                                        // start at root
 6: pred[r] = NIL
 7: Q = newPriQueue(V)                                // put vertices in Q
 8: while Q is nonempty do                            // until all vertices in MST
 9:     u = Q.extraxtMin()                            // lightest edge
10:     for all v ∈ adj[u] do
11:         if (color[v] = W) and (w[u, v] < key[v]) then
12:             key[v] = w[u, v]                      // new lightest edge
13:             Q.decreaseKey(v, key[v])
14:             pred[v] = u
15:         end if
16:     end for
17:     color[u] = B
18: end while
```

Description of Prim's Algorithm

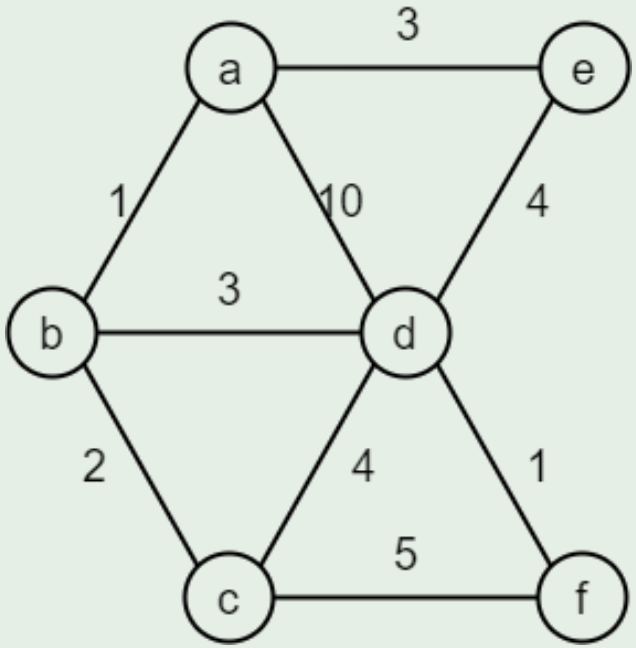
When the algorithm terminates, $Q = \emptyset$ and the MST is

$$T = \{(v, pred[v]) | v \in V \setminus \{r\}\}.$$

- $pred[v]$ defines the predecessor of v.
- And $(v, pred[v])$ is an edge in the MST.

Example for Running Prim's Algorithm

Example



u	a	b	c	d	e	f
$key[u]$						
$pred[u]$						

Outline

Analysis of Prim's Algorithm I

Let $n = |V|$ and $e = |E|$. The data structure *PriQueue* supports the following two operations. (See CLRS)

- $\mathcal{O}(\log n)$ to extract each vertex from the queue. Done once for each vertex $= \mathcal{O}(n \log n)$.
- $\mathcal{O}(\log n)$ time to decrease the key value of neighboring vertex. Done at most once for each edge $= \mathcal{O}(e \log n)$.

Total cost is $\mathcal{O}((n + e) \log n) = \mathcal{O}(e \log n)$.

Analysis of Prim's Algorithm II

Algorithm 2 $Prim(G, w, r)$

```
 1: for all u ∈ V do                                   // O(n)
 2:     key[u] = ∞
 3:     color[u] = W
 4: end for
 5: key[r] = 0                                         // O(1)
 6: pred[r] = NIL                                      // O(1)
 7: Q = newPriQueue(V)                                 // O(n)
 8: while Q is nonempty do                             // For each vertex u
 9:     u = Q.extraxtMin()                             //    O(log n)
10:     for all v ∈ adj[u] do                          //    deg(u)
11:         if (color[v] = W) and (w[u, v] < key[v]) then    // O(1)
12:             key[v] = w[u, v]                       //       O(1)
13:             Q.decreaseKey(v, key[v])               // O(log n)
14:             pred[v] = u                            //       O(1)
15:         end if
16:     end for
17:     color[u] = B                                   //    O(1)
18: end while
```

Analysis of Prim's Algorithm III

So, the overall running time is

$$
\begin{aligned}
& T(n, e) \\
=& \mathcal{O}(n) + \mathcal{O}(1) + \sum_{u \in V} [\mathcal{O}(\log n) + deg(u)(\mathcal{O}(\log n) + \mathcal{O}(1)) + \mathcal{O}(1)] \\
=& \mathcal{O}(n) + \mathcal{O}(1) + \sum_{u \in V} \mathcal{O}(\log n) + \sum_{u \in V} (deg(u)(\mathcal{O}(\log n) + \mathcal{O}(1))) + \sum_{u \in V} \mathcal{O}(1) \\
=& \mathcal{O}(n) + \mathcal{O}(1) + \mathcal{O}(n)\mathcal{O}(\log n) + \mathcal{O}(e)(\mathcal{O}(\log n) + \mathcal{O}(1)) + \mathcal{O}(n) \\
=& \mathcal{O}(n) + \mathcal{O}(1) + \mathcal{O}(n)\mathcal{O}(\log n) + \mathcal{O}(e)\mathcal{O}(\log n) + \mathcal{O}(e)\mathcal{O}(1) + \mathcal{O}(n) \\
=& \mathcal{O}((n + e) \log n) \\
=& \mathcal{O}(e \log n)
\end{aligned}
$$

Note: $\mathcal{O}(n + e) = \mathcal{O}(e)$ because the graph is connected.